

Arreglos en Programación

Taller de Programación I

 Universidad San Sebastián

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$$

Concepto de Arreglos

Un arreglo es una **estructura de datos** que almacena un conjunto de elementos del mismo tipo en ubicaciones **contiguas de memoria**, permitiendo acceder a cada elemento mediante un **índice numérico**.

- ✓ Almacenamiento eficiente de múltiples valores bajo un mismo nombre
- ✓ Acceso directo a elementos mediante índices
- ✓ Base para estructuras de datos más complejas

Arreglos de registros

Arreglo unidimensional de registros (vectores)

Concepto
Es un arreglo cuyo tipo de dato es un registro.

Para el caso de un vector

Vec
1
2
3
4
5
6
7
8

```
CONST
  N = 5;

TYPE
  T_dato = RECORD
    n_registro: string(5);
    nombre: string(40);
    nota: byte;
  END;

  T_vector = ARRAY [1..N] OF T_dato;

VAR
  Vec: T_vector;
```

Inicializar vector de registro

Características de los Arreglos

1

Finita

Todo arreglo tiene un **límite definido** de elementos que puede almacenar.

2

Homogénea

Todos los elementos del arreglo son del **mismo tipo** de dato.

3

Ordenada

Se puede determinar cuál es el **primer, segundo, tercer** elemento, etc.

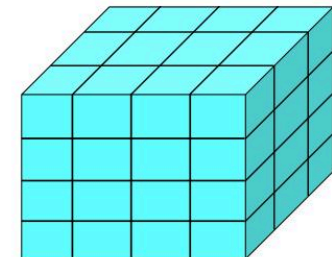
Clasificación de los arreglos

- Los arreglos se clasifican de acuerdo al número de dimensiones que tienen:
 - Unidimensionales (Vectores)
 - Bidimensionales (Matrices o Tablas)
 - Multidimensionales (tres o más dimensiones)

A_0
A_1
A_2
...
A_n

$A_{0,0}$	$A_{0,1}$	...	$A_{0,n}$
$A_{1,0}$	$A_{1,1}$	...	$A_{1,n}$
$A_{2,0}$	$A_{2,1}$	...	$A_{2,n}$
...	...	...	...
$A_{m,0}$	$A_{m,1}$	...	$A_{m,n}$

Clase 8



Tipos de Arreglos



Unidimensional (Vector)

Almacena elementos en **una sola dimensión**, se accede con **un índice**.



Bidimensional (Matriz)

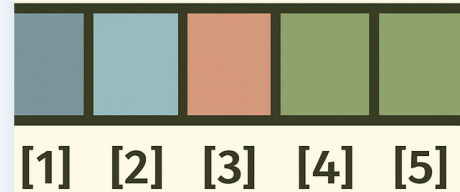
Almacena elementos en forma de **tabla con filas y columnas**, se accede con **dos índices**.



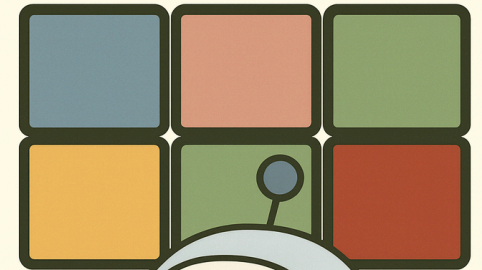
Multidimensional

Almacena elementos en **más de dos dimensiones**, se accede con **múltiples índices**.

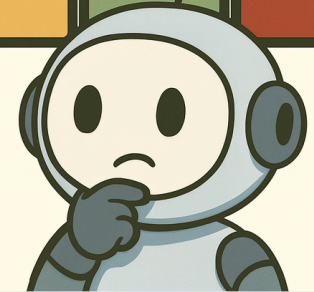
Vector (1D)



Matriz (2D)



Vectores y matrices:
organizando datos paso a paso



Operaciones con Arreglos (Parte 1)



Lectura/Escritura

Proceso para **leer o escribir** elementos en el arreglo.

- ▶ Acceso a **todo el arreglo**
- ▶ Acceso a **elementos específicos** mediante índices



Recorrido

Acceder a **todos los elementos** del arreglo secuencialmente.

- ▶ Utiliza **estructuras repetitivas**
- ▶ Variable de control como **subíndice** del arreglo

Operaciones Básicas con Arreglos

- Búsqueda
- Ordenación
- Inserción
- Borrado

Búsqueda

La búsqueda consiste en encontrar un determinado valor dentro de un conjunto de datos, para recuperar alguna información asociada con el valor buscado.

Existen diferentes formas de hacer esta operación:

- Búsqueda Secuencial o lineal
- Búsqueda Binaria
- Búsqueda HASH
- Árboles de Búsqueda

Búsqueda Secuencial. Es una técnica sencilla para buscar un elemento dentro de un arreglo, y consiste en recorrer todo el vector elemento a elemento desde el primero hasta el último. Si se encuentra el valor buscado, el proceso devolverá la posición donde se encuentra dicho elemento, de lo contrario se enviará un mensaje que informe que el valor buscado no se encuentra dentro del arreglo.

Ejercicio:

Elaborar un subprograma que busque en forma secuencial un VALOR dentro de un arreglo de N elementos.

```
FUNCIÓN BUSQUEDA_SEC (N, VEC, VALORB)
  I ← 1
  MIENTRAS (I <= N) ^ (VALORB <> VEC(I)) HAGA
    I ← I + 1
  FIN MIENTRAS
  SI I <= N ENTONCES
    RETORNE I
  SINO
    RETORNE 0
  FIN SI
FIN BUSQUEDA_SEC
```

Operaciones con Arreglos (Parte 2)



Asignación

Proceso para **asignar valores** a los elementos del arreglo.

- Asignación directa a **posiciones específicas**
- Asignación **masiva** durante inicialización



Actualización

Operaciones para **modificar** los elementos del arreglo.

- + **Añadir:** Verificar espacio disponible
- **Eliminar:** Reorganizar elementos restantes
- ⊕ **Insertar:** Desplazar elementos para hacer espacio



Ventajas y Desventajas de los Arreglos



Ventajas



Acceso rápido a elementos mediante índice



Uso eficiente de la memoria



Fácil recorrido con bucles



Desventajas



Tamaño fijo (a menos que sean dinámicos)



Inserción/eliminación costosa en rendimiento



Un solo tipo de datos por arreglo

75.40 Algoritmos y Programación I Curso 4 Operaciones con Arreglos y Listas

Dr. Mariano Méndez¹

¹Facultad De Ingeniería, Universidad de Buenos Aires

1 de junio de 2020

Correcciones: Manuel Camejo - Martin Dardis - Ignacio Jordan.

Aportes: Gonzalo Hernandez, Valentin Bonani, Andres Fernandez Bina, Nacho, Lucho, Charly Talavera, Javi.

1. Introducción

Existe una serie de operaciones comunes que se realizan entre vectores ya estén compuestos estos por elementos de tipos de datos simples o estructurados. Entre otras operaciones se hallan:

- Insertar elementos en un vector manteniendo un orden.
- Eliminar elementos en un vector.
- Buscar un elemento dentro de un vector.
- Ordenar los elementos de un vector según algún criterio.
- Combinar los elementos de un vector obteniendo un tercer vector:
 - Obtener la Intersección.
 - Obtener la Mezcla.
 - Obtener la Diferencia.
 - Obtener la Unión.
- Actualizar un vector con los datos de otro vector.

Se deberá tener en cuenta que todos los ejercicios están basados en la siguiente declaración:

```
#define MAX 1000
typedef int elemento_t;
typedef elemento_t vector_t[MAX];
```

Aplicaciones Prácticas y Conclusiones



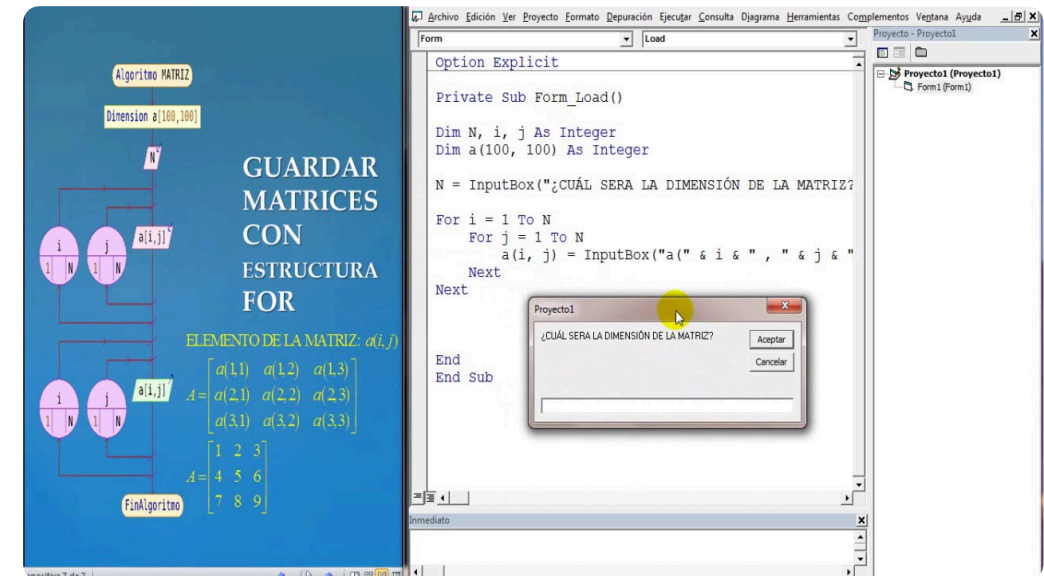
Aplicaciones Prácticas

- Almacenamiento de **grandes volúmenes** de datos
- Optimización de **algoritmos** de búsqueda
- Manejo de **estructuras complejas**
- Procesamiento de **imágenes y señales**
- Desarrollo de **videojuegos**



Conclusiones

- ✓ Estructuras **fundamentales** en programación
- ✓ Permiten **organizar datos** eficientemente
- ✓ Su comprensión es **esencial** para el desarrollo de software
- ✓ Uso adecuado **mejora el rendimiento** de las aplicaciones



★ Punto Clave

Dominar los arreglos es el **primer paso** para entender estructuras de datos más complejas y desarrollar **aplicaciones eficientes**